

Name - Adarsh Singh
Superset Id - **7741812**
VIT Bhopal

PL-SQL

Exercise 1: Control Structures

Code:

```
DECLARE

    CURSOR loan_cursor IS

        SELECT l.LoanID, l.InterestRate, c.DOB

        FROM Loans l

        JOIN Customers c ON l.CustomerID = c.CustomerID;

    v_age NUMBER;
BEGIN

    FOR loan_rec IN loan_cursor LOOP

        -- Calculate age from DOB

        v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, loan_rec.DOB) / 12);

        IF v_age > 60 THEN

            UPDATE Loans

            SET InterestRate = InterestRate - 1

            WHERE LoanID = loan_rec.LoanID;

            DBMS_OUTPUT.PUT_LINE('Loan ID ' || loan_rec.LoanID ||
```

```

        ': Customer age ' || v_age ||
        ' - Discount applied. New rate: ' ||
(loan_rec.InterestRate - 1) || '%');

    ELSE

        DBMS_OUTPUT.PUT_LINE('Loan ID ' || loan_rec.LoanID ||
        ': Customer age ' || v_age || ' - No discount (under
60)');

    END IF;

END LOOP;

COMMIT;

END;

/

-- -----
-- Scenario 2: Set IsVIP flag for customers
-- with balance over $10,000
-- -----

-- Add IsVIP column to Customers table
ALTER TABLE Customers ADD IsVIP VARCHAR2(5) DEFAULT 'FALSE';

DECLARE

    CURSOR customer_cursor IS

        SELECT CustomerID, Name, Balance FROM Customers;

BEGIN

    FOR cust_rec IN customer_cursor LOOP

        IF cust_rec.Balance > 10000 THEN

```

```

        UPDATE Customers

        SET IsVIP = 'TRUE'

        WHERE CustomerID = cust_rec.CustomerID;

        DBMS_OUTPUT.PUT_LINE(cust_rec.Name ||
                               ' (Balance: $' || cust_rec.Balance || ') - Promoted to
VIP');

    ELSE

        UPDATE Customers

        SET IsVIP = 'FALSE'

        WHERE CustomerID = cust_rec.CustomerID;

        DBMS_OUTPUT.PUT_LINE(cust_rec.Name ||
                               ' (Balance: $' || cust_rec.Balance || ') - Not eligible
for VIP');

    END IF;

END LOOP;

COMMIT;

END;

/

-----

-- Scenario 3: Send reminders for loans due
-- within the next 30 days
-- -----

DECLARE

    CURSOR loan_due_cursor IS

        SELECT l.LoanID, l.EndDate, c.Name, c.CustomerID

```

```

        FROM Loans l

        JOIN Customers c ON l.CustomerID = c.CustomerID

        WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30;

v_days_remaining NUMBER;

BEGIN

    DBMS_OUTPUT.PUT_LINE('=====');

    DBMS_OUTPUT.PUT_LINE('    LOAN DUE REMINDERS (Next 30 Days)');

    DBMS_OUTPUT.PUT_LINE('=====');

    FOR loan_rec IN loan_due_cursor LOOP

        v_days_remaining := FLOOR(loan_rec.EndDate - SYSDATE);

        DBMS_OUTPUT.PUT_LINE('');

        DBMS_OUTPUT.PUT_LINE('Reminder for: ' || loan_rec.Name);

        DBMS_OUTPUT.PUT_LINE('Loan ID: ' || loan_rec.LoanID);

        DBMS_OUTPUT.PUT_LINE('Due Date: ' || TO_CHAR(loan_rec.EndDate,
'DD-MON-YYYY'));

        DBMS_OUTPUT.PUT_LINE('Days Remaining: ' || v_days_remaining);

    DBMS_OUTPUT.PUT_LINE('-----');

    END LOOP;

    -- Check if no loans are due

    IF loan_due_cursor%NOTFOUND THEN

        DBMS_OUTPUT.PUT_LINE('No loans due in the next 30 days.');
```

END IF;

END;

/

Output:

```
$ docker exec -i oracle-xe sqlplus -S system/welcome1@localhost/XEPDB1 2>&1 <<'EOF'
ALTER TABLE Customers DROP COLUMN IsVIP;
SET SERVEROUTPUT ON
@/tmp/ControlStructures.sql
EOF

Table altered.

Loan ID 1: Customer age 41 - No discount (under 60)

PL/SQL procedure successfully completed.

PL/SQL procedure successfully completed.

John Doe (Balance: $1000) - Not eligible for VIP
Jane Smith (Balance: $1500) - Not eligible for VIP

PL/SQL procedure successfully completed.

=====
LOAN DUE REMINDERS (Next 30 Days)
=====
No loans due in the next 30 days.

PL/SQL procedure successfully completed.
```

Exercise 2: Stored Procedure

code :

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS

    CURSOR savings_cursor IS

        SELECT AccountID, Balance

        FROM Accounts

        WHERE AccountType = 'Savings';

    v_interest NUMBER;

    v_new_balance NUMBER;

BEGIN

    DBMS_OUTPUT.PUT_LINE('=====');

    DBMS_OUTPUT.PUT_LINE('    PROCESSING MONTHLY INTEREST');

    DBMS_OUTPUT.PUT_LINE('=====');

    FOR acc_rec IN savings_cursor LOOP

        -- Calculate 1% interest

        v_interest := acc_rec.Balance * 0.01;

        v_new_balance := acc_rec.Balance + v_interest;

        -- Update account balance

        UPDATE Accounts

        SET Balance = v_new_balance,

            LastModified = SYSDATE

        WHERE AccountID = acc_rec.AccountID;

        -- Log the transaction
```

```

        INSERT INTO Transactions (TransactionID, AccountID,
TransactionDate, Amount, TransactionType)

        VALUES (

            (SELECT NVL(MAX(TransactionID), 0) + 1 FROM Transactions),

            acc_rec.AccountID,

            SYSDATE,

            v_interest,

            'Interest'

        );

        DBMS_OUTPUT.PUT_LINE('Account ID: ' || acc_rec.AccountID ||

            ' | Previous Balance: $' || acc_rec.Balance ||

            ' | Interest: $' || v_interest ||

            ' | New Balance: $' || v_new_balance);

    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('=====');

    DBMS_OUTPUT.PUT_LINE('Monthly interest processing complete.');
```

END;

/

```

-- -----
-- Scenario 2: Update employee bonus based on
-- department and bonus percentage parameter
-- -----

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(

    p_department IN VARCHAR2,

    p_bonus_percent IN NUMBER
```

```

) AS

CURSOR emp_cursor IS

    SELECT EmployeeID, Name, Salary

    FROM Employees

    WHERE Department = p_department;

v_bonus_amount NUMBER;

v_new_salary NUMBER;

BEGIN

    DBMS_OUTPUT.PUT_LINE('=====');

    DBMS_OUTPUT.PUT_LINE('    UPDATING EMPLOYEE BONUSES');

    DBMS_OUTPUT.PUT_LINE('Department: ' || p_department);

    DBMS_OUTPUT.PUT_LINE('Bonus Percentage: ' || p_bonus_percent ||
'%' );

    DBMS_OUTPUT.PUT_LINE('=====');

    FOR emp_rec IN emp_cursor LOOP

        -- Calculate bonus amount

        v_bonus_amount := emp_rec.Salary * (p_bonus_percent / 100);

        v_new_salary := emp_rec.Salary + v_bonus_amount;

        -- Update employee salary

        UPDATE Employees

        SET Salary = v_new_salary

        WHERE EmployeeID = emp_rec.EmployeeID;

        DBMS_OUTPUT.PUT_LINE('Employee: ' || emp_rec.Name ||

            ' | Current Salary: $' || emp_rec.Salary ||

            ' | Bonus: $' || v_bonus_amount ||

```



```

        ' | New Salary: $' || v_new_salary);

    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('=====');

    DBMS_OUTPUT.PUT_LINE('Bonus update complete.');
```

END;

/

-- -----

-- Scenario 3: Transfer funds between accounts

-- with sufficient balance check

-- -----

```

CREATE OR REPLACE PROCEDURE TransferFunds(

    p_from_account IN NUMBER,

    p_to_account IN NUMBER,

    p_amount IN NUMBER
) AS

    v_source_balance NUMBER;

    v_dest_balance NUMBER;

BEGIN

    DBMS_OUTPUT.PUT_LINE('=====');

    DBMS_OUTPUT.PUT_LINE('    PROCESSING FUND TRANSFER');
```

DBMS_OUTPUT.PUT_LINE('From Account: ' || p_from_account);

DBMS_OUTPUT.PUT_LINE('To Account: ' || p_to_account);

DBMS_OUTPUT.PUT_LINE('Amount: \$' || p_amount);

DBMS_OUTPUT.PUT_LINE('=====');

-- Get source account balance

```

SELECT Balance INTO v_source_balance

FROM Accounts

WHERE AccountID = p_from_account;

-- Get destination account balance

SELECT Balance INTO v_dest_balance

FROM Accounts

WHERE AccountID = p_to_account;

-- Check sufficient balance

IF v_source_balance < p_amount THEN

    DBMS_OUTPUT.PUT_LINE('ERROR: Insufficient balance!');

    DBMS_OUTPUT.PUT_LINE('Available: $' || v_source_balance ||

        ' | Requested: $' || p_amount);

    RETURN;

END IF;

-- Debit source account

UPDATE Accounts

SET Balance = Balance - p_amount,

    LastModified = SYSDATE

WHERE AccountID = p_from_account;

-- Credit destination account

UPDATE Accounts

SET Balance = Balance + p_amount,

    LastModified = SYSDATE

WHERE AccountID = p_to_account;

INSERT INTO Transactions (TransactionID, AccountID, TransactionDate,
Amount, TransactionType)

VALUES (

    (SELECT NVL(MAX(TransactionID), 0) + 1 FROM Transactions),

```

```

        p_from_account,

        SYSDATE,

        p_amount,

        'Withdrawal'

    );

    INSERT INTO Transactions (TransactionID, AccountID, TransactionDate,
Amount, TransactionType)

    VALUES (

        (SELECT NVL(MAX(TransactionID), 0) + 1 FROM Transactions),

        p_to_account,

        SYSDATE,

        p_amount,

        'Deposit'

    );

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Transfer successful!');

    DBMS_OUTPUT.PUT_LINE('Account ' || p_from_account || ' balance: $'
|| (v_source_balance - p_amount));

    DBMS_OUTPUT.PUT_LINE('Account ' || p_to_account || ' balance: $' ||
(v_dest_balance + p_amount));

END;

/

```

Output:

```
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> SET SERVEROUTPUT ON
BEGIN ProcessMonthlyInterest; END;
/
BEGIN UpdateEmployeeBonus('IT', 10); END;
/
BEGIN TransferFunds(1, 2, 200); END;
/SQL> 2 =====
PROCESSING MONTHLY INTEREST
=====
Account ID: 1 | Previous Balance: $424.281 | Interest: $4.24281 | New Balance:
$428.52381
=====
Monthly interest processing complete.

PL/SQL procedure successfully completed.

SQL> 2 =====
UPDATING EMPLOYEE BONUSES
Department: IT
Bonus Percentage: 10%
=====
Employee: Bob Brown | Current Salary: $79860 | Bonus: $7986 | New Salary: $87846
=====
Bonus update complete.

PL/SQL procedure successfully completed.
```